

系统工具开发课程实验报告（第三周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 9 月 7 日

目录

1	实验基本信息	3
2	实验目的	3
3	实验环境与工具	3
4	实验九：从源码构建并在干净环境安装 Wheel	3
4.1	实验要求	3
4.2	包结构	3
4.3	构建与安装验证	4
4.4	结果	4
5	实验十：让编程智能体进入可验证的修复循环	5
5.1	实验要求	5
5.2	失败测试	5
5.3	智能体修复	5
5.4	人工验证与记录	5
6	实验十一：把”无法处理”的协作材料改成可执行信息	6
6.1	实验要求	6
6.2	重写结果	6
6.3	结果	6
7	实验十二：修复一个可复现的线性回归训练循环	7
7.1	实验要求	7
7.2	原始代码与补全	7
7.3	结果	7
8	实验结果汇总	8
9	问题与解决方法	8
9.1	build 模块的 venv 依赖问题	8
9.2	PyTorch 安装问题	8

目录	2
9.3 智能体修改的范围控制	8
10 实验总结	8

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	Linux / Python / build / venv / PyTorch / pytest
实验日期	2026 年 9 月 7 日
GitHub 仓库	https://github.com/MrLLLiao/SystemToolkitDevCourse

2 实验目的

本实验围绕 Python 包打包与分发、智能体编程、协作材料质量改写以及 PyTorch 训练循环修复四个主题展开。通过四个连续实验，掌握从源码构建 wheel 并在干净环境安装的流程，体验编程智能体的测试驱动修复循环，学习将低质量协作材料改写为可执行信息，以及补全 PyTorch 线性回归训练循环。

3 实验环境与工具

本实验在 Linux 和 Windows 环境中完成，主要使用 Python 的 build 模块、venv 虚拟环境、pip 安装、pytest 测试框架、ruff 代码检查工具以及 PyTorch CPU 版本。实验目录为 SystemToolkitDevCourse，四个实验分别位于 q09、q10、q11 和 q12。

4 实验九：从源码构建并在干净环境安装 Wheel

4.1 实验要求

在 q09 中创建一个最小 Python 命令行包 greetlab，包含 src/greetlab/__init__.py、cli.py 和 pyproject.toml。在课程环境中运行 `python -m build` 生成 wheel，新建一个干净虚拟环境只从生成的 wheel 安装，切换到 q09 之外运行 `sdt-greet -name < 学号 >` 确认输出正确。

4.2 包结构

项目目录结构如下：

```
q09/
  pyproject.toml
  src/
    greetlab/
      __init__.py
      cli.py
```

cli.py 实现了一个简单的命令行问候工具：

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

pyproject.toml 配置了构建系统和入口点：

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007073"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.setuptools.packages.find]
where = ["src"]
```

4.3 构建与安装验证

使用 `python -m build --no-isolation` 生成 wheel：

```
cd q09
python -m build --no-isolation
# Successfully built greetlab_25020007073-0.1.0.tar.gz
# and greetlab_25020007073-0.1.0-py3-none-any.whl
```

创建干净虚拟环境并从 wheel 安装：

```
python3 -m venv /tmp/q09_test_venv
source /tmp/q09_test_venv/bin/activate
pip install ./q09/dist/greetlab_25020007073-0.1.0-py3-none-any.whl
```

4.4 结果

在 q09 之外运行 `sdt-greet --name 25020007073` 输出：

```
Hello, 25020007073!
```

确认 wheel 包安装正确，入口点命令 `sdt-greet` 可用。

5 实验十：让编程智能体进入可验证的修复循环

5.1 实验要求

复制 q09 为 q10，添加一个失败测试：当 name 只含空白字符时，main 应以 SystemExit(2) 结束。先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。人工检查 diff，撤销无关修改，并再次运行测试。在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

5.2 失败测试

创建 test_cli.py 包含正常姓名测试和空白姓名测试：

```
import pytest
from greetlab.cli import main

def test_normal_name(capsys):
    import sys
    sys.argv = ["sdt-greet", "--name", "alice"]
    main()
    out = capsys.readouterr().out
    assert "Hello, alice!" in out

def test_blank_name_exits_nonzero():
    import sys
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
```

运行测试确认 test_blank_name_exits_nonzero 失败：

```
FAILED test_blank_name_exits_nonzero - Failed: DID NOT RAISE SystemExit
1 failed, 1 passed
```

5.3 智能体修复

向编程智能体发出提示：修复 greetlab cli.py，当 -name 只含空白字符时 main 应以 SystemExit(2) 退出。约束：不改动参数解析逻辑，测试命令 PYTHONPATH=src pytest test_cli.py。

智能体在 main() 中 parse_args() 后添加了两行：

```
if not a.name.strip():
    raise SystemExit(2)
```

5.4 人工验证与记录

检查 diff，确认仅涉及 cli.py 第 8-9 行，无无关修改。运行 pytest 两项测试均通过。ai_log.md 记录如下：

```
# AI Log
```

提示：修复 `greetlab cli.py`，当 `--name` 只含空白字符时 `main` 应以 `SystemExit(2)` 退出。

智能体改动：在 `main()` 中 `parse_args()` 后添加 `if not a.name.strip(): raise SystemExit(2)`，共增加2行。

人工验证：`diff` 仅涉及 `cli.py` 第8-9行，无无关修改。`pytest 2 passed`，空白姓名测试通过。

6 实验十一：把”无法处理”的协作材料改成可执行信息

6.1 实验要求

围绕”空姓名仍输出问候语”这一缺陷，改写三段质量较差的协作材料：Issue（Windows 上运行不了，尽快修复）、提交信息（fix bug）、评审意见（这里写得不好，重写）。在 `communication.md` 中重写，包含环境、复现命令、期望结果和实际结果，未知信息写”待确认”。全文控制在 400 字以内。

6.2 重写结果

Issue 重写（包含环境、复现命令、期望/实际结果）：

- 标题：sdt-greet `--name` ” ” 输出非空问候语且退出码为 0
- 环境：Windows 11, Python 3.x, greetlab-25020007073 0.1.0
- 复现命令：sdt-greet `--name` ” ”
- 期望结果：当 `name` 只含空白字符时，程序应以非零退出码退出，不输出问候语
- 实际结果：程序输出 Hello, ! 并以退出码 0 结束
- 待确认：是否需要 `name` 做 Unicode 空白字符（如全角空格）的处理

提交信息重写（祈使语气标题，正文说明问题和解决方案）：

```
fix: reject whitespace-only name and exit with code 2
```

```
When --name contains only whitespace, the program previously printed
"Hello, !" and exited 0. Add validation in main() to detect blank
names, print an error message, and exit with SystemExit(2).
```

评审意见重写（标注 Blocking，指出具体行为、风险和建议动作）：

Blocking ——`cli.py:main()` 在 `name` 为纯空白时未做校验，导致输出无意义问候语且退出码为 0，违反 CLI 约定。风险：下游脚本可能依赖退出码判断成功/失败，当前行为会掩盖错误。建议动作：在 `parse_args()` 后增加 `name.strip()` 校验，若为空则 `raise SystemExit(2)`，并补充对应测试用例。

6.3 结果

重写后的三段材料均包含具体的环境信息、复现步骤、期望与实际结果对比，以及明确的修复建议，全文控制在 400 字以内，保持专业、简洁。

7 实验十二：修复一个可复现的线性回归训练循环

7.1 实验要求

将给定的 PyTorch 线性回归代码保存为 q12/train.py，补全两个 TODO：清空梯度、反向传播、更新参数；进入评估模式并在 no_grad 中打印最终损失、weight 和 bias。最终损失应小于 0.001，不得直接把 weight 和 bias 赋值为 3 和 -1。

7.2 原始代码与补全

原始代码中训练循环缺少梯度清空、反向传播和参数更新步骤：

```
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数
    # TODO: 进入评估模式并在no_grad中打印最终损失、weight和bias
```

补全后的完整训练循环：

```
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_pred = model(x)
    final_loss = loss_fn(final_pred, y)
    print(f"final_loss={final_loss.item():.6f}")
    print(f"weight={model.weight.item():.6f}")
    print(f"bias={model.bias.item():.6f}")
```

7.3 结果

运行 train.py 输出：

```
final_loss=0.000000
weight=2.999997
bias=-1.000000
```

最终损失为 0.000000，小于 0.001 的要求。weight 接近 3，bias 接近 -1，与目标函数 $y = 3x - 1$ 一致。训练过程通过 SGD 优化器（学习率 0.1）在 200 轮迭代中收敛，未直接赋值参数。

表 2: 四个实验的完成情况

题号	实验主题	验证结果
9	打包与安装 Wheel	wheel 构建成功, 干净环境安装并运行正确
10	智能体修复循环	失败测试确认, 智能体修复后 2 项测试通过
11	协作材料改写	Issue/提交/评审三段材料均改写为可执行信息
12	PyTorch 训练循环	补全 zero_grad/backward/step, 损失 < 0.001

8 实验结果汇总

9 问题与解决方法

9.1 build 模块的 venv 依赖问题

在 Windows Python 中运行 `python -m build` 时, 由于缺少 `venv` 模块导致构建失败。解决方法是使用 `-no-isolation` 参数跳过隔离环境创建, 直接使用当前 Python 环境中的 `setuptools` 进行构建。

9.2 PyTorch 安装问题

WSL 环境中的 Python 3.14 没有预装 PyTorch, 且 `externally-managed-environment` 限制了全局安装。解决方法是在 Windows Python 中通过 CPU 专用索引安装 PyTorch (`pip install torch --index-url https://download.pytorch.org/whl/cpu`), 安装后成功运行训练脚本。

9.3 智能体修改的范围控制

在实验十中, 需要确保智能体的修改仅限于必要范围。通过人工检查 diff, 确认修改仅涉及 `cli.py` 中新增的两行校验代码, 没有对参数解析逻辑或其他文件产生无关变更。

10 实验总结

通过本次实验, 我完整体验了从源码打包到 wheel 分发的流程, 理解了 `pyproject.toml` 中构建系统、入口点和包发现的配置方式。实验十让我认识到编程智能体在测试驱动修复中的价值: 只要提供明确的目标、约束和测试命令, 智能体可以完成最小化修复, 但仍需人工审查 diff 以确保不引入无关变更。

实验十一强化了协作材料的质量意识: Issue 应包含可复现的环境和命令, 提交信息应使用祈使语气并说明“为什么”, 评审意见应标注严重级别并给出具体建议。实验十二则让我实践了 PyTorch 训练循环的标准流程: `zero_grad` → `backward` → `step` 的三步序列, 以及评估模式下使用 `no_grad` 防止梯度计算的正确做法。

本周的核心认识是: 无论是包安装、智能体修复还是模型训练, “可验证性”都是关键——wheel 需要在干净环境中验证安装, 修复需要通过测试验证, 训练需要通过损失值验证收敛。